

Table Extraction Pipeline - Detailed Documentation

Github Repository : [Link](#)

Project Overview

Introduction

The **Table Extraction Pipeline** is an end-to-end automated system designed to extract structured data from PDF-based sports record books. This pipeline processes large volumes of **scanned and digital PDFs**, detects tables, matches them with the unsan.txt file and merges extracted information with an existing stagebook records database (**MongoDB**). It operates on an **asynchronous task execution model** using **FastAPI, Celery, Redis, and AWS Textract**.

Objectives

- Automate **table extraction** from record book PDFs.
 - Use **AWS Textract** to extract tables from images.
 - **Match extracted tables** with reference tables stored in **MongoDB**.
 - Store and retrieve data efficiently in **AWS S3 buckets**.
 - Provide **real-time task execution monitoring** using **Flower**.
 - Deploy the pipeline using **Docker and Docker-Compose**.
-

System Architecture

The system is built on **microservices architecture** with multiple components interacting asynchronously.

UML code and png format of diagrams is available in github repository

High-Level Design (HLD)



Low-Level Design (LLD)



Technology Stack

Component	Technology Used
Web Framework	FastAPI
Asynchronous Tasks	Celery
Message Broker	Redis
Storage (o/p & i/p)	AWS S3
OCR/Extraction	AWS Textract
Database for SBR	MongoDB
Monitoring	Flower
Containerization	Docker, Docker-Compose

SBR - Stagebook Records

Pipeline Flow

1. **User submits a job via FastAPI endpoint**
 2. **Input files are downloaded from AWS S3**
 3. **PDF is converted to images**
 4. **Tables are extracted using AWS Textract**
 5. **Extracted tables are matched against MongoDB records**
 6. **Matched records are merged and saved in MongoDB & S3**
 7. **Progress is tracked and stored in a JSON file**
 8. **Final results are uploaded back to S3**
 9. **Users can monitor job status via Flower UI and Progress Route**
-

Project Structure

```
Unset
RECORDBOOK_EXTRACTION/
|-- app/
|   |-- routes/                # FastAPI route handlers
|   |   |-- submit_job.py      # Handles job submission and
|   |   tracking               #
|   |   |-- job_status.py      # Retrieves job status
|   |   |-- utils/             # Utility scripts for
|   |   processing             #
|   |   |-- s3_utils.py        # AWS S3 interactions
|   |   |-- pdf_processing.py  # Converts PDFs to images
|   |   |-- progress_tracker.py # Tracks and updates
|   |   processing progress
|   |   |-- mongo_utils.py     # MongoDB interactions
|   |   |-- table_extraction.py # Table extraction and
|   |   matching
|   |   |-- celery_worker.py   # Celery worker for
|   |   background jobs
|   |   |-- main.py            # FastAPI app entry point
|   |   |-- tasks.py           # Celery task definitions
|-- docker/
|   |-- Dockerfile             # Dockerfile for
|   containerizing the app
|-- docker-compose.yml         # Docker services (Web,
|   Redis, MongoDB, Worker, Flower)
|-- README.md                  # Project Documentation
|-- requirements.txt            # Python Dependencies
```

Pipeline Breakdown

Step 1: Job Submission

- Users submit a job to process a **college dataset**
- **Either name of the college or the name of the folder which has colleges's folders**
- The API forwards this request to a Celery worker

Step 2: Data Processing (Celery Worker) and output of each stage

1. **Download files from S3** (PDF, config.json, unsan.txt)
2. **Convert PDF to images**
 - a. (/images)
3. **Process each image** (Textract OCR, table extraction,)
 - a. saved in CSV_dump/textract_tables
 - b. all_tables_drawn/ (folder)
4. **Match extracted tables with reference data** (unsan.txt)
 - a. saved in matched_unmatched_json_files - page wise
 - b. matched_tables_drawn/ - page wise
5. **Combine matched tables from JSON files into a master output**
 - a. (master_json_file.json)
6. **Merge each table of Master json file to the corresponding stagebook records using 'Headers' in MongoDB (if found)** (Merged_json_file.json)
7. **Upload all the processed results back to S3** (s3 bucket in output-data/ folder)

Step 3: Progress Tracking

1. The system updates **progress.json** periodically
2. No input required for this - just 'Execute' Command
3. Users can fetch progress via **/progress/**
4. **Exhaustive Chronological List of Monitoring Steps (User View)**
 - a. This is the complete step-by-step progress tracking that the user will see in real-time while processing the folder.

Folder-Level Progress (process_college())

- b. Downloading Input Files → Fetching files from S3 (record-book.pdf, unsan.txt, config.json).
- c. PDF to Image Conversion → Converting PDF to images.
- d. Processing Images (X/Y) → Progress updates as each image is processed.
Image-Level Progress (process_single_image()) (For each image inside college_name/images/ folder, these steps will repeat:)
- e. Textract Processing → Running AWS Textract on the image.
- f. Processing Unsan File → Extracting tables from unsan.txt.
- g. Extracting Headers → Fetching headers from unsan.txt.
- h. BLEU Score Comparison → Comparing extracted tables with reference tables using BLEU score.
- i. Saving Results → Storing matched/unmatched tables in CSV and JSON.

Finalization Steps (process_college())

- j. Combining JSON Files → Merging all matched table JSON files into master_json_file.json.
- k. Uploading Results to S3 → Uploading processed data to S3.
- l. Completed → Processing finished for the college.

Step 4: Job Status Retrieval

- Users can check job completion using **/job-status/{job_id}**

- Use the `job_id` you get after making the POST request to process a specific college or a folder of colleges.
- Process info is stored and retrieved from S3

API Endpoints

1. Submit a Job

- **POST** `/submit-job/`
- **Description:** Submits a job for processing a single college.
- **Request Body:**

Unset

```
{
  "s3_access_key": "your_access_key",
  "s3_secret_key": "your_secret_key",
  "s3_region_name": "your_region",
  "textract_access_key": "your_textract_key",
  "textract_secret_key": "your_textract_secret",
  "textract_region_name": "your_textract_region",
  "bucket_name": "recordbooks",
  "folder_name": "input-data",
  "college_name": "college_1"
}
```

- **Response:**

Unset

```
{
  "job_id": "abc123",
  "status": "Job submitted"
}
```

2. Process Entire Folder

- **POST** `/process-folder/`
- **Description:** Processes all colleges inside an S3 folder.
- **Request Body:**

Unset

```
{
  "s3_access_key": "your_access_key",
  "s3_secret_key": "your_secret_key",
  "s3_region_name": "your_region",
  "textract_access_key": "your_textract_key",
  "textract_secret_key": "your_textract_secret",
  "textract_region_name": "your_textract_region",
  "bucket_name": "recordbooks",
  "folder_name": "input-data",
  "college_name": "college_1"
}
```

- **Response:**

Unset

```
{
  "job_id": "abc123",
  "status": "Job submitted"
}
```

3. Get Progress

- **GET** `/progress/`
- **Response Example:**
- **No input required for this**

Unset

```
{
  "current_college": "college_1",
  "progress": {
    "college_1": {
      "stage": "BLEU Score Comparison",
      "images_processed": "12/20"
    }
  }
}
```

4. Get Job Status

- **GET** `/job-status/{job_id}`
- **Request Body**

Unset

```
{  
  "job_id": "id_from_POST_request",  
  "status": "Job submitted"  
}
```

- **Response**

Unset

```
{  
  "job_id": "abc123",  
  "status": "COMPLETED"  
}
```

Setup & Installation

Clone the Repository

Unset

```
git clone <repository-url>  
cd recordbook_extraction
```

Install Dependencies

Unset

```
pip install -r requirements.txt
```

Start Services

Unset

```
docker-compose up --build
```

Debugging & Monitoring

View Celery Worker Logs

Unset

```
docker-compose logs -f worker
```

Access Flower Monitoring Dashboard

Unset

```
http://localhost:5555
```

List Running Containers

Unset

```
docker ps
```

Access Redis from Inside Docker

Unset

```
docker exec -it redis redis-cli
```

Input & Output Directory Structure

Input Data (AWS S3 Bucket `s3://recordbooks/input-data/`)

Unset

```
input-data/
├── college_1/
│   ├── record-book.pdf
│   ├── unsan.txt
│   └── config.json
├── college_2/
│   └── record-book.pdf
```

```
|   ├── unsan.txt
|   ├── config.json
```

Output Data (AWS S3 Bucket `s3://recordbooks/output-data/`)

```
Unset
output-data/
|— college_1/
|   ├── images/
|   ├── all_tables_drawn/
|   ├── matched_tables_drawn/
|   ├── matched_unmatched_json_files/
|   ├── CSV_dump/
|   |   ├── textract_tables/
|   |   ├── matched_tables/
|   |   └── unmatched_tables/
|   ├── master_json_file.json
|   ├── merged_json_file.json
|   └── matching_evaluation.txt
```

Future Enhancements

- Optimize for GPUs and stronger processors
 - Add more sophisticated table matching techniques
 - Implement alert notifications for errors (Slack/Email)
 - Optimize MongoDB queries for better performance
-